

Article model. Django will automatically create the table to link between tags and articles and use it to fetch tags for an article and the other way around.

You'll be interested to know that it is also possible to have One to Many / Many to One and Many to Many relationships within the same model / data! For instance employees can have a boss, who is an employee as well. There is a many to one relationship here since many employees can share the same boss. Likewise, if we track friendships between employees, then one employee can have many other employees as friends, which is a many to many relationship from employees to employees.

There is one other kind of relationship possible between data, and that is a One to One relationship. An example of this could be an employee profile. A single employee will only have a single profile and that profile will belong exclusively to that employee. Of course it would be possible to just have the profile data included within the employee table itself, however sometimes this can be useful for efficiency, lesser used and accessed data can be kept off in a separate table.

Testing out our new Model

Now that we have a new Model class in our application we can see test how it works in the Django shell. The Django shell is a convenient tool provided by 'manage.py' that simply loads a Python interpreter that is properly wired to work with your Django project. You can start this shell by executing 'python3 manage.py shell'

Remember though, what we mentioned in our 'Starting with Django' chapter. After any change to the models, you need to run 'python3 manage.py makemigrations' followed by 'python3 manage.py migrate'

Now that you are in the Django shell you need to import your Article model to use. You can do this by typing:

- `from cms.models import Article`

Now you can use the `Article` class to access the list of articles and to create new Articles. Since the database is currently empty, let's create a new Article. In the shell run the following commands, which will create a new blank article, fill in some of the details and then save it to the database:

- `>>> article1 = Article()`
- `>>> article1.title = "New Article"`
- `>>> article1.content = "Test content"`
- `>>> article1.slug = "new-article"`
- `>>> article1.save()`